

Module M214

Application Cloud Native

TP Projet de Groupe

Architecture Microservices

avec Express.js et MongoDB

Organisation du TP

- **Travail en groupe** : 4 à 6 stagiaires par groupe
- **Présentation** : 10 minutes par groupe
- **Livrable** : Code source + Documentation + Démonstration

OFPPT

Développement Digital - Option Full Stack

1 Introduction

1.1 Objectifs du TP

Ce TP a pour objectif de mettre en pratique les concepts d'architecture microservices en réalisant un projet complet en équipe. Chaque membre du groupe sera responsable de l'implémentation d'un microservice.

Compétences visées

- Concevoir une architecture microservices à partir d'un besoin métier
- Définir les APIs et les communications inter-services
- Implémenter des microservices avec Express.js et MongoDB
- Travailler en équipe avec une répartition des tâches
- Intégrer et tester une application distribuée
- Présenter et défendre un travail technique

1.2 Formation des groupes

(!) Consignes importantes

- Formez des groupes de **4 à 6 stagiaires**
- Chaque groupe doit avoir un **chef de projet** désigné
- Le nombre de microservices doit être **égal ou supérieur** au nombre de membres
- Chaque membre sera responsable d'**au moins un microservice**

Information	À compléter
Nom du groupe	
Chef de projet	
Membres du groupe	
Nombre de membres	

2 Exercice : Projet Microservices en Groupe

ÉTAPE 1 : Choix et définition du projet

Objectif : Choisir un projet adapté à une architecture microservices et définir ses fonctionnalités.

Consignes :

1. Choisissez un projet qui peut être **naturellement divisé** en plusieurs services indépendants
2. Le projet doit avoir suffisamment de fonctionnalités pour justifier **autant de microservices que de membres**
3. Définissez clairement le **périmètre** du projet et ses **fonctionnalités principales**

Exemples de projets éligibles

- **E-commerce** : Produits, Commandes, Utilisateurs, Paiements, Notifications, Livraison
- **Réservation hôtelière** : Hôtels, Chambres, Réservations, Clients, Paiements, Avis
- **Livraison de repas** : Restaurants, Menus, Commandes, Livreurs, Clients, Paiements
- **Gestion d'école** : Étudiants, Professeurs, Cours, Notes, Emplois du temps, Absences
- **Location de voitures** : Véhicules, Agences, Réservations, Clients, Factures
- **Réseau social** : Utilisateurs, Publications, Commentaires, Messages, Notifications

À compléter :

Nom du projet : _____

Description du projet :

Liste des fonctionnalités principales :

Nř	Fonctionnalité
1	
2	
3	
4	
5	
6	
7	
8	

ÉTAPE 2 : Définition des microservices

Objectif : Identifier et définir les microservices de l'application.

Rappel :

- **Service Fonctionnel (Métier)** : Gère une fonctionnalité métier principale (ex : Produits, Commandes, Clients)
- **Service Utilitaire** : Fournit des services transversaux (ex : Authentification, Notifications, Paiements)

Consignes :

1. Le nombre de microservices doit être $\geq$ **nombre de membres du groupe**
2. Pour chaque service, définissez son **type** (Fonctionnel ou Utilitaire)
3. Listez les **fonctionnalités** de chaque microservice
4. Attribuez un **port** unique à chaque service

À compléter :

Microservice	Type	Port	Fonctionnalités
	F / U		
	F / U		
	F / U		
	F / U		
	F / U		
	F / U		

F = Fonctionnel (Métier) / U = Utilitaire

ÉTAPE 3 : Définition des bases de données

Objectif : Définir la structure de la base de données de chaque microservice.

Rappel : Chaque microservice possède sa **propre base de données** (principe d'autonomie).

À compléter pour chaque microservice :

Microservice 1 : _____	
Nom de la BDD	
Modèle(s) / Schéma(s)	

Microservice 2 : _____	
Nom de la BDD	
Modèle(s) / Schéma(s)	

Microservice 3 : _____	
Nom de la BDD	
Modèle(s) / Schéma(s)	

ÉTAPE 3 (suite) : Bases de données**Microservice 4 :** _____

Nom de la BDD	
Modèle(s) / Schéma(s)	

Microservice 5 : _____

Nom de la BDD	
Modèle(s) / Schéma(s)	

Microservice 6 : _____

Nom de la BDD	
Modèle(s) / Schéma(s)	

ÉTAPE 4 : Définition des APIs et communications

Objectif : Définir les endpoints de chaque microservice et les communications inter-services.

Pour chaque API, précisez :

- La **méthode HTTP** (GET, POST, PUT, DELETE)
- L'**endpoint** (URL)
- La **description** / usage de l'API
- Si l'API a besoin d'appeler un **autre service** (dépendance)
- Le **type de communication** : Synchrone (S) ou Asynchrone (A)

Microservice 1 : _____

Méthode	Endpoint	Description	Dépendance	S/A

Microservice 2 : _____

Méthode	Endpoint	Description	Dépendance	S/A

S = Synchrone / A = Asynchrone / Dépendance = Nom du service appelé ou "Aucune"

ÉTAPE 4 (suite) : APIs et communications**Microservice 3 :** _____

Méthode	Endpoint	Description	Dépendance	S/A

Microservice 4 : _____

Méthode	Endpoint	Description	Dépendance	S/A

Microservice 5 : _____

Méthode	Endpoint	Description	Dépendance	S/A

ÉTAPE 4 (suite) : APIs et communications**Microservice 6 :** _____

Méthode	Endpoint	Description	Dépendance	S/A

Schéma d'architecture (à dessiner) :

Dessinez le schéma de l'architecture montrant les microservices, leurs bases de données et les communications entre eux.

ÉTAPE 5 : Attribution et implémentation

Objectif : Répartir les microservices entre les membres et les implémenter.

Consignes :

1. Chaque membre se voit attribuer **au moins un microservice**
2. Chaque membre est **responsable** de son service (code, tests, documentation)
3. Utilisez **Express.js** avec **Mongoose** pour MongoDB
4. Respectez les spécifications définies dans les étapes précédentes

Attribution des microservices :

Membre	Microservice(s) attribué(s)	Port

(!) Rappel technique**Structure recommandée pour chaque microservice :**

```
nom-service/  
  config/  
    db.js          # Connexion à MongoDB  
  controllers/  
    controller.js  # Logique métier  
  middlewares/  
    authMiddleware.js # Middleware d'authentification  
    errorMiddleware.js # Gestion des erreurs  
  models/  
    Model.js       # Schémas Mongoose  
  routes/  
    routes.js      # Définition des routes  
  utils/  
    helpers.js     # Fonctions utilitaires  
    validators.js  # Validation des données  
  index.js         # Point d'entrée  
  package.json  
  .env             # Variables d'environnement
```

Packages à installer :

- express - Framework web
- mongoose - ODM MongoDB
- axios - Client HTTP (si communication inter-services)
- dotenv - Variables d'environnement
- nodemon - Rechargement automatique
- jsonwebtoken - Authentification JWT
- bcryptjs - Hashage des mots de passe
- cors - Cross-Origin Resource Sharing

ÉTAPE 6 : Tests individuels des microservices

Objectif : Tester chaque microservice individuellement avant l'intégration.

Consignes :

1. Utilisez **Postman** (ou un outil similaire comme Insomnia, Thunder Client)
2. Testez **tous les endpoints** de votre microservice
3. Documentez les **requêtes et réponses**
4. Vérifiez les **codes de statut HTTP** (200, 201, 400, 404, 500...)

Grille de tests pour chaque microservice :

Endpoint	Méthode	Statut	OK ?	Observations

Codes HTTP courants

- **200 OK** : Requête réussie
- **201 Created** : Ressource créée avec succès
- **400 Bad Request** : Erreur dans la requête
- **401 Unauthorized** : Non authentifié
- **404 Not Found** : Ressource non trouvée
- **500 Internal Server Error** : Erreur serveur

ÉTAPE 7 : Intégration et tests globaux

Objectif : Regrouper tous les microservices et tester l'application complète.

Consignes :

1. Regroupez tous les codes sources dans un **dossier commun**
2. Vérifiez que chaque service utilise un **port différent**
3. Lancez **tous les services simultanément**
4. Testez les **scénarios complets** impliquant plusieurs services
5. Vérifiez les **communications inter-services**

Structure recommandée du projet global :

```
projet-microservices/  
  auth-service/  
  produit-service/  
  commande-service/  
  notification-service/  
  paiement-service/  
  client-service/  
  README.md
```

Checklist d'intégration :

Vérification	OK ?
Tous les services démarrent sans erreur	
Chaque service est accessible sur son port	
Les communications synchrones fonctionnent	
Les communications asynchrones fonctionnent (si applicable)	
Les scénarios métier complets fonctionnent	
Aucune erreur de connexion aux bases de données	
Les tokens JWT sont validés correctement (si applicable)	

Scénarios de test à valider :

ÉTAPE 8 : Présentation du projet

Objectif : Présenter le travail réalisé devant la classe.

Durée : 10 minutes par groupe (incluant questions/réponses)

Plan de présentation suggéré :

N°	Contenu	Durée
1	Présentation du projet et de l'équipe	1 min
2	Architecture choisie (schéma des microservices)	2 min
3	Démonstration de l'application	4 min
4	Difficultés rencontrées et solutions	1 min
5	Questions / Réponses	2 min

(!) Rappel des livrables

À la fin de la séance, chaque groupe doit fournir :

1. Le **code source** de tous les microservices
2. La **documentation** (ce document complété)
3. Une **collection Postman** avec tous les tests
4. Le **schéma d'architecture** du projet